FPT University

HO CHI MINH CITY CAMPUS



Embedded Linux Systems

Assignment 1:

How to Write a Device Driver

ioctl Interface | Misc Device | Driver Analysis | ARMv7 Target

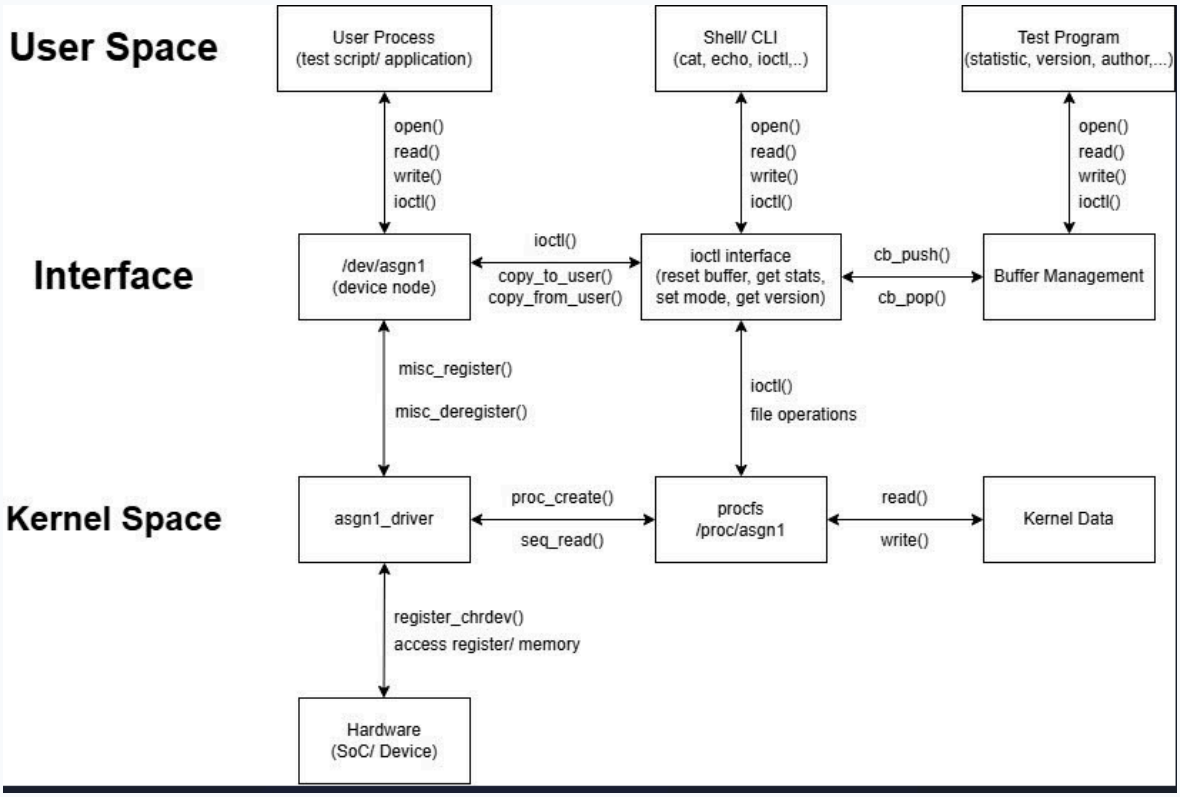
Lecturer: Phạm Thế Vinh

Name	Roll number	Subject
Hoàng Ngọc Gia Bão	SE203238	LOS201
Trần Hồ Khánh Tân	SE203740	LOS201
Nguyễn Trọng Hùng	SE204352	LOS201
Nguyễn Quế Vũ	SE204290	LOS201
Nguyễn Việt Anh	SE204364	LOS201
Hồ Phan Anh Tuấn	SE203437	LOS201

Ho Chi Minh City, Oct 14, 2026

1. SYSTEM ARCHITECTURE & PROJECT STRUCTURE

This assignment implements two Linux kernel drivers for the ARMv7 vexpress-a9 target emulated by QEMU. The first, asgn1_driver, extends the LAB-02 character driver with a four-command ioctl interface, an internal buffer, operation counters and two operating modes. The second, counter_driver, is an independent misc device that exposes an atomic hardware-style counter through /dev/counter and /proc/counter_info. Both drivers are exercised from userspace by shell test scripts and a small ioctl test program.



[FIGURE 1] architecture_diagram.png — *High-Level Architecture and Data Flow of the Assignment 01 driver stack (userspace, system calls, kernel driver layers, ioctl commands and driver structure)*

The deliverable is organized with one folder per task, a logs folder holding the raw terminal output and dmesg, and the report folder, following the submission layout required by the handout:

1.1. Driver Interface Specifications

Aspect	asgn1_driver (Task 1)	counter_driver (Task 2)
Registration	register_chrdev(), fixed major in 241–254, node created with mknod	struct miscdevice + misc_register() / misc_deregister(), major 10 (MISC_MAJOR)
Device node	/dev/asgn1	/dev/counter
Interfaces	open / read / write / release / unlocked_ioctl, /proc/asgn1_info	open / read / write / release, /proc/counter_info
State	Internal buffer, open/write/read counters, last_write_size, mode	Single atomic_t counter, read and write totals
Synchronization	Mutex around every buffer and counter access	atomic_read / atomic_set / atomic_inc
User ↔ kernel copy	copy_to_user(), copy_from_user() (no memcpy)	copy_to_user(), kstrtoint() on a copied string
Test script	test_asgn1.sh (ioctl test program inside QEMU)	test_counter.sh

1.2. ioctl Command Specifications

Command	Direction	Argument	Behavior
`ASGN1_RESET_BUFFER`	_IO	none	Clears the internal buffer, resets write_count and read_count to 0; cat /dev/asgn1 then returns an empty string
`ASGN1_GET_STATS`	_IOR	struct asgn1_stats	Returns open_count, write_count, read_count, buffer_len, last_write_size (and mode) with copy_to_user()
`ASGN1_SET_MODE`	_IOW	int	Mode 0 (Kernel): prepends [KERNEL] to written data. Mode 1 (Echo, default): stores data unchanged. Other values return -EINVAL
`ASGN1_GET_VERSION`	_IOR	char[64]	Returns the version string, bounded to the 64-byte userspace buffer

Any command that does not match one of the four cases returns -ENOTTY, and every ioctl call writes an identifying line to the kernel log.

2. CORE MODULE IMPLEMENTATION & IOCTL INTERFACE (TASK 1)

2.1. Driver Design

asgn1_driver is organized in three layers that are kept separate in the source: an interface layer (file_operations and unlocked_ioctl dispatch), an operation layer (mode handling, prefixing and statistics) and a data layer (buffer, counters, version string). The implementation satisfies the mandatory technical requirements:

- **asgn1_ioctl.h** is a standalone header shared by the kernel module and the userspace test. It defines the magic number, struct asgn1_stats and the four commands with _IO, _IOR and _IOW.
- **unlocked_ioctl** handles all four commands inside one switch and returns -ENOTTY from the default branch.
- **A mutex** protects every read or write of the buffer and counters, so concurrent open() and write() calls cannot corrupt the state.
- **copy_to_user() / copy_from_user()** move the stats struct, the mode value and the version string across the user boundary. The version copy is limited to the 64-byte userspace buffer.
- **dmesg logging:** open(), release(), write() and each ioctl command print one explicit line, and module unload prints a cleanup message.

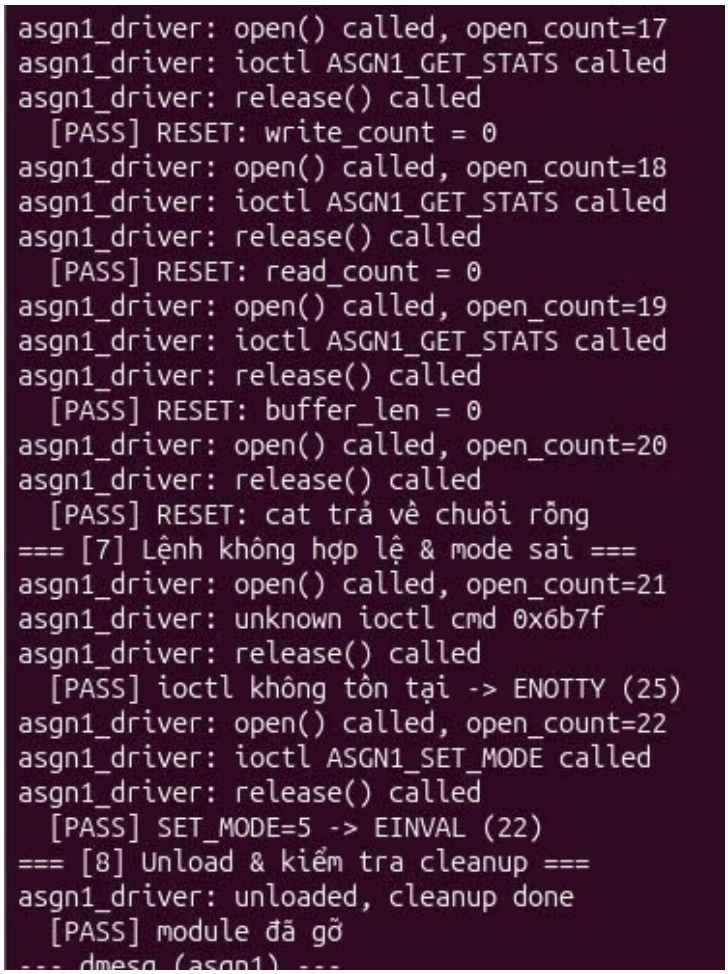
2.2. Functional Test in QEMU

test_asgn1.sh runs inside QEMU (vexpress-a9, Cortex-A9, 512 MB, initramfs). It loads the module, creates /dev/asgn1, writes and reads back in both modes, calls all four ioctl commands, verifies RESET, sends invalid inputs and finally unloads the module. Evidence 01 shows the GET_STATS, GET_VERSION and RESET_BUFFER steps, where the driver log lines interleave with the test output.



[EVIDENCE 01] 01_task1_ioctl_test_stats_version_reset.png — *Terminal output of test_asgn1.sh in QEMU: ASGN1_GET_STATS, ASGN1_GET_VERSION and ASGN1_RESET_BUFFER steps*

The remaining steps cover the post-reset checks, the error paths and the unload check. The test finishes with 17 passed and 0 failed.



[EVIDENCE 02] 02_task1_ioctl_test_errors_cleanup.png — *Terminal output of test_asgn1.sh: RESET verification, invalid ioctl and mode, module unload, dmesg cleanup and final result*

Table 1 summarizes what the captured logs verify.

Test step	Observed result	Verdict
GET_STATS after 3 writes and 3 reads	open_count=10, write_count=3, read_count=3, buffer_len=4, last_write_size=4, mode=1	PASS (every field checked)
GET_VERSION	asgn1_driver v1.0 - SE203437	PASS (version string correct)
SET_MODE back to Echo	Stored data has no [KERNEL] prefix	PASS
RESET_BUFFER after a 16-byte write	write_count, read_count and buffer_len read back as 0; cat returns an empty string	PASS (4 checks)
Unknown ioctl cmd 0x6b7f	Driver logs "unknown ioctl cmd 0x6b7f"; ioctl returns ENOTTY (25)	PASS
SET_MODE = 5	ioctl returns EINVAL (22)	PASS

Test step	Observed result	Verdict
rmmod and dmesg	"unloaded, cleanup done" printed; module no longer listed	PASS
Final result	17 passed, 0 failed	ALL PASS

TASK 1 VERDICT: All four ioctl commands behave as specified. Direction macros match the data flow of each command, invalid commands and values are rejected with the correct errno, and the driver releases all resources on unload.

3. MISC DEVICE DRIVER: COUNTER (TASK 2)

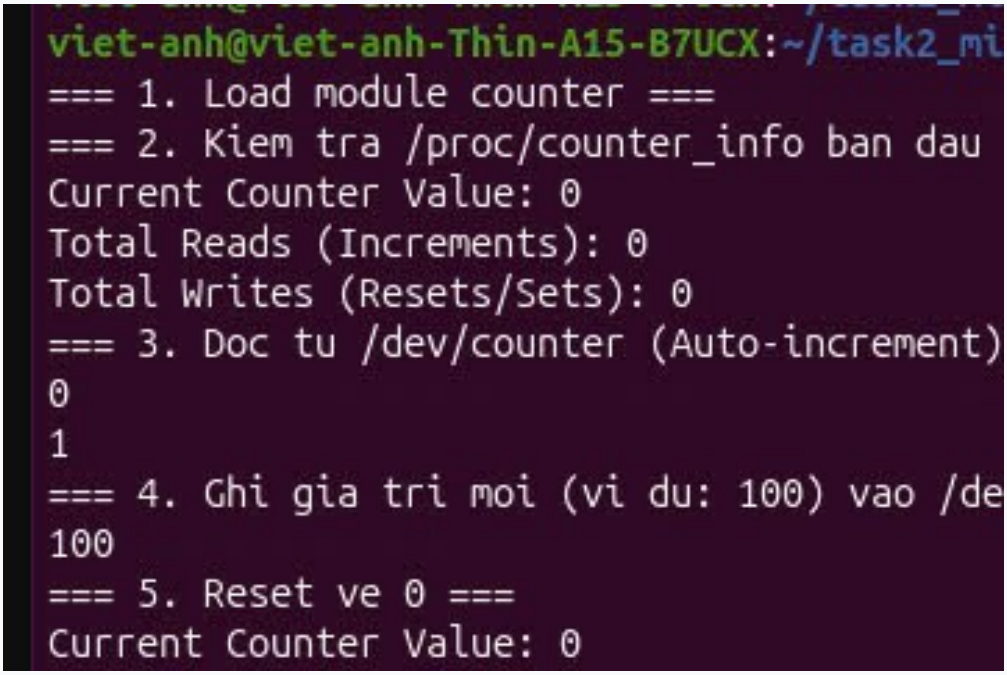
3.1. Driver Design

counter_driver is a new module, independent of Task 1. It models a simple hardware counter and is registered through the misc device framework, so the kernel assigns major 10 and only the minor number has to be chosen.

- **Registration:** a static struct miscdevice named "counter" is passed to misc_register() in init and to misc_deregister() in exit. /dev/counter exists after insmod.
- **read():** returns the current value as a decimal string with a trailing newline and then increments the counter (auto-increment on read). The *ppos offset is handled so a second read() call returns 0 and cat terminates.
- **write():** copies the user string, parses it with kstrtoint() and sets the counter. Writing "0" resets it; negative or non-numeric input returns -EINVAL.
- **Thread safety:** the counter is an atomic_t accessed with atomic_read(), atomic_set() and atomic_inc().
- **/proc/counter_info:** prints the current value, total reads (increments) and total writes (resets/sets), each on a labelled line.
- **Module info:** MODULE_LICENSE, MODULE_AUTHOR (name and student ID) and MODULE_DESCRIPTION are set.

3.2. Functional Test

The test script loads the module, prints /proc/counter_info before any access, reads /dev/counter twice, writes 100, reads it back, writes 0 to reset, prints /proc/counter_info again and unloads the module.



[EVIDENCE 03] 03_task2_counter_test.png — *Terminal output of the counter_driver test: /proc/counter_info before and after the test, auto-increment reads, set to 100 and reset to 0*

Step	Observed output	Verdict
Initial /proc/counter_info	Current Counter Value: 0, Total Reads: 0, Total Writes: 0	PASS
Two reads of /dev/counter	0 then 1 (auto-increment)	PASS
Write 100, then read	Read returns 100	PASS
Write 0 (reset)	Current Counter Value: 0	PASS
Final /proc/counter_info	Total Reads: 3, Total Writes: 2	PASS (matches 3 reads, 2 writes)
Unload module	Test completes without error	PASS

TASK 2 VERDICT: The device registers correctly, read and write follow the specification, and the statistics in /proc/counter_info stay consistent with the number of operations performed.

4. DRIVER DESIGN ANALYSIS

4.1. Character vs Block vs Network Devices

Criterion	Character device	Block device	Network device
VFS interface	file_operations	block_device_operations	net_device_ops (no device

Criterion	Character device	Block device	Network device
		(plus request queue / blk-mq)	node, accessed via sockets)
Data unit	Byte stream	Block (sector, typically 512 B or 4 KB)	Packet (frame)
Buffering	Usually none; the driver may add its own buffer or FIFO	Page cache and buffer cache, I/O scheduler, read-ahead	sk_buff queues, DMA rings, NAPI polling
Embedded examples	UART, GPIO, I2C/SPI sensors, /dev/counter	eMMC, SD card, USB mass storage	Ethernet MAC, Wi-Fi, CAN
Access pattern	Sequential or ioctl-controlled	Random access by sector	Asynchronous send and receive

Design decisions. (1) A temperature sensor on I2C delivers a few bytes per sample with no filesystem need, so a character driver with read() and ioctl() is the natural fit. (2) An eMMC chip holds a filesystem that needs random sector access, caching and an I/O scheduler, so it must be a block device. (3) A CAN controller exchanges framed messages with addressing and error handling, so it belongs to the network stack rather than to a byte-stream device node.

4.2. register_chrdev() vs misc_register()

	register_chrdev() (Task 1)	misc_register() (Task 2)
Major / minor	Driver claims its own major (241–254 here) and the minors under it	Shared major 10; only a minor is needed (static or MISC_DYNAMIC_MINOR)
Device node	Created manually with mknod in the test script	Created automatically by devtmpfs / udev
Boilerplate	More code; the modern cdev + class API needs even more	One struct and one call
Pros	Full control of major and minor range; supports many minors per driver	Simple, fewer errors, no wasted major number
Cons	Manual major management and node creation, reserves 256 minors	Limited to single-purpose devices; misc minor space is small and shared
Use when	A driver needs several minors or its own major	Small single-device drivers, common in embedded systems

Task 1 uses register_chrdev() because the handout requires a hand-chosen major number and a mknod-based test. Task 2 uses the misc framework because a single counter node needs no more than a minor number.

4.3. Why counter_driver Uses atomic_t

counter++ on a plain int compiles to load, add, store. If two processes read /dev/counter on different CPUs at the same time, both can load the value 5, both compute 6 and both store 6. One increment is lost, and the two readers may even receive the same number. atomic_inc() uses an indivisible read-modify-write (ldrex/strex on ARMv7), so each increment takes effect exactly once without needing a lock. A caveat: atomic_read() followed by atomic_inc() is two operations, so a strict "return and increment" would use atomic_fetch_inc() or atomic_inc_return().

4.4. ioctl Magic Number and Command Encoding

The magic number identifies the driver that owns a command. Without it, a command number such as 1 would be accepted by any driver, and a wrongly opened file descriptor could trigger an unintended operation. A command is a 32-bit value built by _IOC(dir, type, nr, size):

Bits 31–30	Bits 29–16	Bits 15–8	Bits 7–0
dir (_IOC_DIR)	size (_IOC_SIZE), 14 bits	type = magic (_IOC_TYPE)	nr = command number (_IOC_NR)

_IOC_NR(cmd) is (cmd >> 0) & 0xFF and _IOC_SIZE(cmd) is (cmd >> 16) & 0x3FFF. For ASGN1_GET_VERSION, defined as _IOR(ASGN1_MAGIC, 4, char[64]), the direction is read, _IOC_SIZE is 64 and _IOC_NR is 4. The invalid command 0x6b7f in Evidence 02 decodes as direction 0, size 0, type 0x6b ('k') and nr 0x7f, i.e. _IO('k', 127). It matches none of the switch cases, so the driver returns -ENOTTY.

5. THEORY QUESTIONS (TASK 4)

Question 1. Module reference counting.

The kernel increments a module's reference count with try_module_get() when something starts using it, for example when open() succeeds on a device whose file_operations has .owner = THIS_MODULE, and decrements it with module_put() when the file is released. The count appears in the "Used by" column of lsmod. If rmmod is run while a process holds the device open, the count is non-zero and the call fails with "Module is in use" (-EBUSY). The clean fix is to close or kill the process. Forced unload (rmmod -f) works only if the kernel has CONFIG_MODULE_FORCE_UNLOAD; it ignores the count, taints the kernel and risks a crash because code that is still referenced disappears.

Question 2. Why copy_to_user() and not memcpy().

A userspace pointer is untrusted: it may be unmapped, point into kernel memory, or refer to a page that is swapped out. copy_to_user() first checks the range with access_ok() and performs the copy through exception-table fixups, so a bad address makes it return the number of bytes not copied (the caller returns -EFAULT) instead of crashing. A direct memcpy() relies on the MMU translating that address with the current process's page tables; if the page is absent or protected, the page-fault handler cannot resolve it in kernel context and the kernel oopses. It would also bypass the hardware protection (SMAP/PAN or ARM domain-based protection) that stops the kernel from touching user memory by accident, and let a malicious pointer write into kernel memory.

Question 3. MTD vs block device.

The block layer assumes a device whose sectors can be rewritten in place at random, with the controller hiding the medium's details. Raw flash has different rules: it is erased in large blocks, a bit can only be cleared until the block is erased, blocks wear out, and NAND has bad blocks and needs ECC. The MTD subsystem exposes these operations (read, write, erase, bad block handling) and lets flash-aware software such as UBI/UBIFS or JFFS2 handle wear levelling and bad blocks. A storage device needs MTD when it is raw flash without an internal translation layer, such as raw NAND or SPI/parallel NOR flash holding the bootloader, kernel and rootfs on a board. eMMC and SD cards contain their own flash translation layer and are used through the block layer.

Question 4. Handling *ppos in read().

VFS calls read() through vfs_read(), passing &file->f_pos as ppos. A positive return value means that many bytes were delivered and f_pos advances; cat keeps calling read() and stops only when read() returns 0 (EOF). If counter_driver ignored *ppos and always returned the number string, every call would return a positive count, cat would never see EOF, and it would print numbers forever while the counter increments each time. With the check, read() returns the string and sets *ppos to its length on the first call; on the next call, *ppos > 0 so it returns 0 and cat exits. A new open() starts with f_pos = 0, so each new cat gets a fresh value.

6. EXPERIMENTAL EVIDENCE MATRIX

Table 2 cross-references all experimental artifacts captured for this report.

Evidence ID	Filename	Validation Procedure / Command	Status
FIGURE 1	architecture_diagram.png	High-level architecture and data flow of the driver	VERIFIED (Complete)

Evidence ID	Filename	Validation Procedure / Command	Status
		stack	
EVIDENCE 01	01_task1_ioctl_test_stats_version_reset.png	./test_asgn1.sh in QEMU: GET_STATS, GET_VERSION, RESET_BUFFER	VERIFIED (Passed)
EVIDENCE 02	02_task1_ioctl_test_errors_cleanup.png	./test_asgn1.sh in QEMU: invalid ioctl, invalid mode, rmmod and dmesg (17 passed, 0 failed)	VERIFIED (Passed)
EVIDENCE 03	03_task2_counter_test.png	sudo ./test.sh: counter_driver load, read, write, reset, /proc/counter_info	VERIFIED (Passed)

7. CONCLUSION & ENGINEERING EVALUATION

Both tasks were implemented and verified. asgn1_driver provides a complete ioctl interface in which the command direction, the copy helpers and the error codes (-ENOTTY, -EINVAL) match the data flow of each command, and the 17 automated checks pass. counter_driver shows the misc device framework in its simplest form, with an atomic counter, correct *ppos handling and consistent statistics in procfs.

Through this assignment the main skills practiced were designing ioctl commands with the _IO macro family, moving data safely across the user–kernel boundary, protecting shared state with a mutex or atomic operations, choosing between register_chrdev() and the misc framework, and verifying kernel modules with scripted tests and dmesg on an ARMv7 QEMU target.